

第 12 节 实操任务 2 编写 Eval 评估测试

目标: pytest tests/eval_test.py 通过 3+ 个评估用例

目标文件: tests/eval_test.py

背景

AI 系统的输出不确定——同一个输入，每次调用可能得到不同的输出。传统的 `assert output == expected` 行不通。

Eval 测试的核心思路：不测精确内容，测行为边界。

- 用 `>=`，`<=`，`in` 代替 `==`
- 测试“有没有摘要”而非“摘要内容是什么”
- 正面 + 负面 + 边界 = 最小 Eval 集

步骤 1：用 AI 编程工具生成 eval_test.py

以下代码可以用 OpenCode、Claude Code、Cursor、Trae 或通义灵码等任意 AI 编程工具生成。

提示词：

```
1 请帮我在 v3-multi-agent/tests/ 下编写 eval_test.py, 实现 AI 知识库的评估测试 :
2
3 需求 :
4 1. 使用 pytest 框架
5 2. 文件顶部用 dotenv 加载 .env, 让 pytest 能读到 LLM_API_KEY ;
6    并屏蔽 PytestUnknownMarkWarning (避免自定义 slow 标记触发警告)
7 3. 定义 EVAL_CASES 列表, 至少包含 3 种场景 :
8    - 正面案例 : 技术文章输入, 预期有摘要、有关键词
9    - 负面案例 : 无关内容输入, 预期被过滤或标记为低相关
10   - 边界案例 : 极短输入 (如 "AI" ), 预期不崩溃
11 4. 每个用例包含 : name, input, expected (含检查函数或范围条件)
12 5. 测试函数使用范围断言 ( >=, <=, in ), 不用精确匹配
13 6. 包含一个 LLM-as-Judge 测试 :
14   - 让 LLM 对分析结果打分 (1-10)
15   - 断言分数 >= 5
16 7. 标记 LLM 测试为 pytest.mark.slow (可选跳过)
17 8. 包含一个不调用 LLM 的本地验证测试 (验证 EVAL_CASES 结构)
18
19 依赖 : workflows/model_client.py 的 chat() 函数
```

```
20 chat(prompt, system=...) 返回 (text, usage)
```

生成的代码（参考实现）：

```
1  """Eval 评估测试 - AI 知识库质量验证
2
3  核心原则：
4  - 不测精确内容，测行为边界
5  - 用 >=, <=, in 代替 ==
6  - 正面 + 负面 + 边界 = 最小 Eval 集
7  - LLM-as-Judge 做质量评分
8  """
9
10 import json
11 import os
12 import sys
13 import warnings
14
15 import pytest
16 from dotenv import load_dotenv
17
18 # 确保可以导入 workflows 模块
19 PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
20 sys.path.insert(0, PROJECT_ROOT)
21
22 # 加载 .env, 让 pytest 能读到 LLM_API_KEY
23 load_dotenv(os.path.join(PROJECT_ROOT, ".env"))
24
25 # 屏蔽 "Unknown pytest.mark.slow" 警告 (slow 是自定义慢速标记)
26 warnings.filterwarnings("ignore", category=pytest.PytestUnknownMarkWarni
27
28 from workflows.model_client import chat
29
30
31 # — 评估用例定义 —————
32
33 EVAL_CASES = [
34     {
35         "name": "正面案例 - 技术项目分析",
36         "input": "LangGraph 是一个基于有向图的多 Agent 工作流编排框架，支持条件:
37         "expected": {
38             "min_length": 50,
39             "max_length": 1000,
40             "must_contain_any": ["LangGraph", "工作流", "Agent", "图"],
41         },
42     },
43     {
44         "name": "负面案例 - 无关内容",
45         "input": "今天天气真好，适合出去野餐，带上三明治和果汁。",
```

```

46         "expected": {
47             "max_length": 500,
48             "should_mention_irrelevant": True,
49         },
50     },
51     {
52         "name": "边界案例 - 极短输入",
53         "input": "AI",
54         "expected": {
55             "min_length": 1,
56             "no_crash": True,
57         },
58     },
59     {
60         "name": "正面案例 - 英文技术内容",
61         "input": "OpenAI released GPT-5 with 1M token context window and",
62         "expected": {
63             "min_length": 30,
64             "must_contain_any": ["GPT-5", "OpenAI", "token", "context"],
65         },
66     },
67 ]
68
69
70 # — 本地验证 (不调 LLM) —————
71
72 def test_eval_cases_structure():
73     """验证 EVAL_CASES 的结构完整性 (不消耗 token) """
74     assert len(EVAL_CASES) >= 3, "至少需要 3 个评估用例"
75
76     names = [c["name"] for c in EVAL_CASES]
77     assert any("正面" in n for n in names), "缺少正面案例"
78     assert any("负面" in n for n in names), "缺少负面案例"
79     assert any("边界" in n for n in names), "缺少边界案例"
80
81     for case in EVAL_CASES:
82         assert "name" in case, f"用例缺少 name 字段"
83         assert "input" in case, f"用例 {case.get('name')} 缺少 input"
84         assert "expected" in case, f"用例 {case.get('name')} 缺少 expected"
85
86
87 # — LLM 评估测试 (消耗 token) —————
88
89 @pytest.mark.slow
90 def test_eval_positive():
91     """正面案例：技术内容应生成有意义的分析"""
92     case = EVAL_CASES[0]
93     prompt = f"请分析以下技术内容，输出 200 字以内的中文摘要：\n{case['input']}"

```

```

94     result, usage = chat(prompt, system="你是技术分析师。")
95
96     expected = case["expected"]
97     assert len(result) >= expected["min_length"], f"输出太短: {len(result)}"
98     assert len(result) <= expected["max_length"], f"输出太长: {len(result)}"
99
100    found = any(kw in result for kw in expected["must_contain_any"])
101    assert found, f"输出应包含以下关键词之一: {expected['must_contain_any']}"
102
103
104    @pytest.mark.slow
105    def test_eval_negative():
106        """负面案例：无关内容应被识别"""
107        case = EVAL_CASES[1]
108        prompt = f"请判断以下内容是否与 AI 技术相关，如果不相关请说明：\n{case['input']}"
109        result, usage = chat(prompt, system="你是技术内容筛选器。")
110
111        # 负面案例：输出应提及"不相关"或类似表述
112        assert result is not None and len(result) > 0
113
114
115    @pytest.mark.slow
116    def test_eval_boundary():
117        """边界案例：极短输入不应崩溃"""
118        case = EVAL_CASES[2]
119        try:
120            result, usage = chat(f"请分析：{case['input']}", system="你是技术分
121            assert result is not None
122            assert len(result) > 0
123        except Exception as e:
124            pytest.fail(f"边界输入不应导致崩溃: {e}")
125
126
127    @pytest.mark.slow
128    def test_llm_as_judge():
129        """LLM-as-Judge：让 LLM 对分析质量打分"""
130        # 先生成分析结果
131        analysis, _ = chat(
132            "请分析 LangGraph 框架的核心优势和适用场景",
133            system="你是技术分析师。输出 Markdown 格式。",
134        )
135
136        # 让另一个 LLM 调用评分
137        judge_prompt = f"""请对以下技术分析的质量打分（1-10分）。
138
139        分析内容：
140        {analysis}
141

```

```

142 评分标准：
143 - 准确性：信息是否正确
144 - 深度：是否有洞察
145 - 实用性：读者能否据此行动
146
147 只返回一个数字（1-10），不要解释。"""
148
149     score_text, _ = chat(judge_prompt, system="你是质量评审。只返回数字。", t
150
151     try:
152         score = int(score_text.strip())
153     except ValueError:
154         # 提取数字
155         import re
156         match = re.search(r"\d+", score_text)
157         score = int(match.group()) if match else 5
158
159     assert 1 <= score <= 10, f"评分应在 1-10 范围内，实际：{score}"
160     assert score >= 5, f"分析质量评分过低：{score}/10"
161     print(f"LLM-as-Judge 评分：{score}/10")
162
163
164 # — 运行入口 —————
165
166 if __name__ == "__main__":
167     print("=== 本地验证（不消耗 token）===")
168     test_eval_cases_structure()
169     print(f"[OK] EVAL_CASES 结构验证通过，共 {len(EVAL_CASES)} 个用例")
170     for c in EVAL_CASES:
171         print(f"    - {c['name']}")
172
173     print("\n提示：运行 LLM 测试请使用：")
174     print("    pytest tests/eval_test.py -m slow -v")
175
176
177 ---

```

步骤 2：理解代码

如果你对这段代码有疑问，可以让 AI 编程工具解释：

请解释 eval_test.py 的设计：

1. 为什么用 must_contain_any 而不是 must_contain_all？
2. pytest.mark.slow 有什么作用？
3. LLM-as-Judge 的原理是什么？可靠吗？
4. 为什么 test_eval_cases_structure 不调 LLM？

关键设计解读：

设计点	为什么这样做
must_contain_any	LLM 输出不确定，只要包含相关词之一就算合格
pytest.mark.slow	LLM 测试消耗 token，标记后可选择性跳过
LLM-as-Judge	用 LLM 评判 LLM 的输出，比规则断言更灵活
本地验证	不消耗 token 就能验证测试框架本身是否正确

步骤 3：运行测试

```
1 cd ~/ai-knowledge-base
2
3 # 先跑本地验证（不消耗 token）
4 pytest tests/eval_test.py -v
5
6 # 再跑完整 Eval（消耗 token）
7 pytest tests/eval_test.py -m slow -v
```

期望输出（本地验证）：

```
1 === 本地验证（不消耗 token）===
2 [OK] EVAL_CASES 结构验证通过，共 4 个用例
3   - 正面案例 - 技术项目分析
4   - 负面案例 - 无关内容
5   - 边界案例 - 极短输入
6   - 正面案例 - 英文技术内容
7
```

完整 Eval：

```
huangj2@Levono-V2:~/ai-knowledge-base/v3-multi-agent$ pytest tests/eval_test.py -v
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.0.2, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/huangj2/ai-knowledge-base/v3-multi-agent
plugins: langsmith-0.7.11, anyio-4.13.0
collected 17 items

tests/eval_test.py::TestArticleQuality::test_summary_quality PASSED [ 5%]
tests/eval_test.py::TestArticleQuality::test_article_format PASSED [ 11%]
tests/eval_test.py::TestArticleQuality::test_tag_format PASSED [ 17%]
tests/eval_test.py::TestReviewLoop::test_max_iteration_forced_pass PASSED [ 23%]
tests/eval_test.py::TestReviewLoop::test_iteration_increments PASSED [ 29%]
tests/eval_test.py::TestReviewLoop::test_feedback_propagation PASSED [ 35%]
tests/eval_test.py::TestCostGuard::test_basic_tracking PASSED [ 41%]
tests/eval_test.py::TestCostGuard::test_budget_exceeded PASSED [ 47%]
tests/eval_test.py::TestCostGuard::test_alert_threshold PASSED [ 52%]
tests/eval_test.py::TestCostGuard::test_report_generation PASSED [ 58%]
tests/eval_test.py::TestSecurity::test_injection_detection PASSED [ 64%]
tests/eval_test.py::TestSecurity::test_clean_input_passes PASSED [ 70%]
tests/eval_test.py::TestSecurity::test_pii_detection PASSED [ 76%]
tests/eval_test.py::TestSecurity::test_pii_detection_only PASSED [ 82%]
tests/eval_test.py::TestSecurity::test_rate_limiter PASSED [ 88%]
tests/eval_test.py::TestSecurity::test_audit_logger PASSED [ 94%]
tests/eval_test.py::TestEndToEnd::test_full_pipeline |
```

步骤 4：提交到 Git

```
1 cd ~/ai-knowledge-base/v3-multi-agent
```

```
2 git add tests/eval_test.py
3 git commit -m "feat: add eval test suite with positive/negative/boundary
4
5 ---"
```

完成！ Eval 测试覆盖了正面、负面、边界三种场景，加上 LLM-as-Judge 质量评分。范围断言适应 AI 输出的不确定性。